

Sales Data Warehouse & Analytics

Design, Implementation & ETL Project Report

Medallion Architecture on Microsoft SQL Server

Namit

Portfolio Project

github.com/Namitt/data-warehouse-project

Contents

1. Executive Summary	3
2. Objectives & Scope	3
2.1 Objectives	3
2.2 In Scope	3
2.3 Out of Scope	3
3. Solution Architecture	4
3.1 Database & Schema Initialisation	4
4. Source Data	5
4.1 Source Files	5
5. ETL Implementation	5
5.1 Bronze Layer: Raw Ingestion	5
5.2 Silver Layer: Cleansing & Standardisation	6
crm_cust_info — customer core	6
crm_prd_info — product master	6
crm_sales_details — sales transactions	7
erp_cust_az12 — supplementary customer data	7
erp_loc_a101 — customer location	7
erp_px_cat_g1v2 — product categories	7
5.3 Gold Layer: Dimensional Model	8
gold.dim_customers	8
gold.dim_products	8
gold.fact_sales	8
6. Data Model	9
7. Data Quality Issues Resolved	9
8. Validation & Testing	10
9. Future Work	10
10. Conclusion	10

1. Executive Summary

This report documents the design and implementation of a sales data warehouse built on Microsoft SQL Server. The project consolidates customer, product, and sales data originating from two separate source systems, a CRM application and an ERP application, delivered as CSV extracts, into a single dimensional model suitable for analytical reporting.

The warehouse follows the medallion architecture, organising data into three progressive layers. The Bronze layer ingests the source files exactly as received. The Silver layer applies cleansing, standardisation, and derivation logic to produce trustworthy, analysis-ready tables. The Gold layer exposes a star schema of dimension and fact views that integrate the two sources behind clean, business-friendly names.

The project was undertaken as a structured learning exercise to build practical, end-to-end data engineering skills: SQL-based ETL, dimensional modelling, and data quality handling. It was implemented by following an established tutorial methodology and then documented, validated, and version-controlled as an independent portfolio piece. The sections that follow describe the architecture, the source data, the transformation logic applied at each layer, the resulting data model, and the data quality issues resolved along the way.

Scope is deliberately bounded: the build targets the latest snapshot of the data only, with no historisation requirement, and concludes at a query-ready Gold layer. Analytics and reporting are identified as the next phase.

2. Objectives & Scope

2.1 Objectives

- Consolidate sales data from two disparate source systems (CRM and ERP) into a single, integrated model.
- Apply systematic data cleansing and standardisation so that downstream analysis can be trusted.
- Design a dimensional star schema optimised for analytical querying.
- Produce clear documentation, data model, data catalog, naming conventions, to support future users.
- Establish a repeatable, re-runnable ETL process driven by stored procedures.

2.2 In Scope

- Ingestion of six source CSV files (three CRM, three ERP).
- Full-load ETL across Bronze and Silver layers, and view-based Gold layer.
- One customer dimension, one product dimension, and one sales fact.

2.3 Out of Scope

- Historisation / slowly changing dimensions, only the current snapshot is modelled.
- Incremental loading, every run performs a full truncate-and-insert.

- Production concerns such as orchestration scheduling, security hardening, and performance tuning at scale.
- The BI / reporting layer, which is identified as a follow-on phase.

3. Solution Architecture

The warehouse is structured into three layers within a single SQL Server database (DataWarehouse), each isolated in its own schema: bronze, silver, and gold. Data moves strictly in one direction, source files into Bronze, Bronze into Silver, Silver into Gold; so that each layer has a single, well-defined responsibility.

Layer	Object Type	Load Method	Responsibility
Bronze	Tables	Full load (truncate & insert) via BULK INSERT	Hold raw source data exactly as received; the traceable source of truth.
Silver	Tables	Full load (truncate & insert) via INSERT...SELECT	Cleansed, standardised, and derived data still mapped to source entities.
Gold	Views	None - computed on read from Silver	Business-ready star schema integrating both sources for analytics.

The choice of object type per layer is deliberate. Bronze and Silver are physical tables because their data must be stored and validated. Gold is implemented as views, so the business model is always computed from the latest Silver data without maintaining a separate physical copy eliminating the risk of the analytical layer drifting out of sync with the cleaned data beneath it.

3.1 Database & Schema Initialisation

A single initialisation script provisions the environment: it drops and recreates the DataWarehouse database, then creates the three schemas. Re-creating the database from scratch guarantees a clean, reproducible starting state on every full rebuild, appropriate for a snapshot-only project where no history needs preserving.

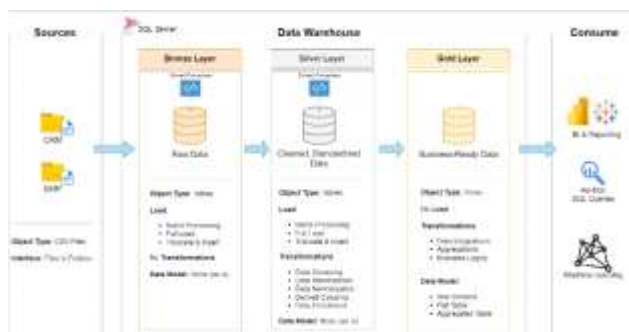


Figure 1: Data Architecture

4. Source Data

Data originates from two systems, each contributing three CSV files. The CRM system is the primary source for customer identity, product master data, and sales transactions. The ERP system supplements customer records with demographic and geographic detail and provides the product category hierarchy.

4.1 Source Files

Source	File / Bronze Table	Contents
CRM	crm_cust_info	Core customer records: ID, key, name, marital status, gender, create date.
CRM	crm_prd_info	Product master: ID, key, name, cost, product line, start/end dates.
CRM	crm_sales_details	Sales transactions: order number, product key, customer ID, dates, sales, quantity, price.
ERP	erp_cust_az12	Supplementary customer data: customer ID, birthdate, gender.
ERP	erp_loc_a101	Customer location: customer ID, country.
ERP	erp_px_cat_g1v2	Product category hierarchy: ID, category, subcategory, maintenance flag.

A notable structural detail drives part of the transformation logic: the CRM sales table stores order, ship, and due dates as integers (e.g. 20101229) rather than as date types, and the CRM product key encodes the ERP category ID as its first five characters. Both facts are handled explicitly in the Silver layer.

5. ETL Implementation

5.1 Bronze Layer: Raw Ingestion

The Bronze layer is loaded by the stored procedure `bronze.load_bronze`. For each of the six tables it truncates the target and then bulk-loads the corresponding CSV using `BULK INSERT`, skipping the header row and treating the comma as field terminator. No transformation is applied; the goal is a faithful copy of the source.

Operational features built into the procedure:

- Per-table and total load durations are captured with `GETDATE()` and printed, giving basic load observability.
- The whole batch is wrapped in `TRY...CATCH`, so any failure is reported with the SQL error message, number, and state rather than failing silently.
- `TABLOCK` is used on each bulk insert to improve load performance into the empty target.

Because Bronze tables are defined to mirror the source exactly, several columns are intentionally the "wrong" type at this stage, most visibly, the three sales dates are stored as `INT`, matching the

raw file. Correcting these is the Silver layer's job, preserving the principle that Bronze never transforms.

```

=====
Loading Bronze Layer
=====
Loading CRM Tables
-----
>> Truncating Table: bronze.crm_cust_info
>> Inserting Data Into: bronze.crm_cust_info

(18453 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: bronze.crm_prd_info
>> Inserting Data Into: bronze.crm_prd_info

(297 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: bronze.crm_sales_details
>> Inserting Data Into: bronze.crm_sales_details

(60398 rows affected)
>> Load Duration: 0 seconds
>> -----
Loading ERP Tables
-----
>> Truncating Table: bronze.erp_loc_a101
>> Inserting Data Into: bronze.erp_loc_a101

(18484 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: bronze.erp_cust_as12
>> Inserting Data Into: bronze.erp_cust_as12

(18483 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: bronze.erp_px_cat_glv2
>> Inserting Data Into: bronze.erp_px_cat_glv2

(37 rows affected)
>> Load Duration: 0 seconds
>> -----
Loading Bronze Layer is Completed
- Total Load Duration: 0 seconds
=====

Completion time: 2026-06-19T18:58:49.7631927+01:00

```

Figure 2: EXEC bronze.load_bronze

5.2 Silver Layer: Cleansing & Standardisation

The Silver layer, loaded by silver.load_silver, is where the substantive data quality work happens. Each target table is truncated and repopulated from its Bronze counterpart through an INSERT...SELECT that embeds the cleansing logic. The transformations applied to each table are described below; they represent the core engineering contribution of the project.

crm_cust_info — customer core

- Deduplication: a ROW_NUMBER() window partitioned by customer ID and ordered by create date descending keeps only the most recent record per customer, discarding stale duplicates.
- Null filtering: records with a null customer ID are excluded entirely.
- Whitespace cleaning: first and last names are TRIM-med.
- Standardisation: single-letter marital status and gender codes (S/M, F/M) are expanded to readable values (Single/Married, Female/Male), with anything unrecognised mapped to 'n/a'.

crm_prd_info — product master

- Key splitting: the product key is decomposed, the first five characters become *cat_id* (with dashes converted to underscores) to join to the ERP category table, and the remainder becomes the actual product key.
- Null handling: missing product cost is defaulted to 0 via ISNULL.
- Standardisation: product-line codes (M/R/S/T) are mapped to descriptive names (Mountain, Road, Other Sales, Touring).
- Derived end date: each product's end date is calculated as one day before the next start date for the same key, using *LEAD()* turning overlapping records into clean, non-overlapping validity periods.

crm_sales_details — sales transactions

- Date repair: the integer order, ship, and due dates are validated (rejecting zero or non-8-digit values as NULL) and otherwise cast through VARCHAR into proper DATE values.
- Sales recalculation: where the recorded sales amount is null, non-positive, or inconsistent with $\text{quantity} \times |\text{price}|$, it is recomputed as $\text{quantity} \times \text{ABS}(\text{price})$, correcting bad or missing totals.
- Price derivation: where price is null or non-positive, it is derived as $\text{sales} \div \text{quantity}$ (guarding against divide-by-zero with NULLIF).

erp_cust_az12 — supplementary customer data

- Key normalisation: a leading 'NAS' prefix is stripped from the customer ID so it aligns with the CRM key.
- Future-date handling: birthdates later than today are nulled out as invalid.
- Gender standardisation: assorted gender spellings (F/FEMALE, M/MALE) are normalised to Female/Male, else 'n/a'.

erp_loc_a101 — customer location

- Key normalisation: dashes are removed from the customer ID to match the CRM key format.
- Country standardisation: codes are expanded (DE → Germany, US/USA → United States) and blank or null countries are set to 'n/a'.

erp_px_cat_g1v2 — product categories

This table arrives clean and is passed through to Silver unchanged, beyond being reloaded under the standardised process. It carries the metadata column described below.

Every Silver table also carries a `dwh_create_date` metadata column, defaulted to the load timestamp, distinguishing warehouse-generated bookkeeping from genuine source fields, consistent with the project's naming convention for technical columns.

```

=====
Loading Silver Layer
=====
Loading CRM Tables
>> Truncating Table: silver.crm_cust_info
>> Inserting Data Into: silver.crm_cust_info
(18484 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: silver.crm_prd_info
>> Inserting Data Into: silver.crm_prd_info
(397 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: silver.crm_sales_details
>> Inserting Data Into: silver.crm_sales_details
(60338 rows affected)
>> Load Duration: 1 seconds
>> -----
>> Truncating Table: silver.erp_cust_addr
>> Inserting Data Into: silver.erp_cust_addr
(18484 rows affected)
>> Load Duration: 0 seconds
>> -----
Loading ERP Tables
>> Truncating Table: silver.erp_loc_addr
>> Inserting Data Into: silver.erp_loc_addr
(18484 rows affected)
>> Load Duration: 0 seconds
>> -----
>> Truncating Table: silver.erp_cat_giv2
>> Inserting Data Into: silver.erp_cat_giv2
(27 rows affected)
>> Load Duration: 0 seconds
>> -----
Loading Silver Layer is Completed
- Total Load Duration: 1 seconds
=====
Completion time: 2026-06-19T17:01:57.4403031+01:00

```

Figure 3: EXEC Silver.load_silver

5.3 Gold Layer: Dimensional Model

The Gold layer is expressed entirely as views over the Silver tables, assembling the star schema on read. Three views are created.

gold.dim_customers

- Integrates the CRM customer core with ERP demographic and location data via two left joins on the shared customer key.
- Generates a surrogate key with *ROW_NUMBER()*.
- Resolves gender by treating CRM as the system of record: the CRM value is used unless it is 'n/a', in which case it falls back to the ERP value, then to 'n/a'. This is a genuine integration decision, made because the two systems can disagree.

gold.dim_products

- Joins the Silver product master to the ERP category table on the derived category ID, enriching each product with category, subcategory, and maintenance flag.
- Generates a surrogate product key, and filters to current products only with *WHERE prd_end_dt IS NULL* — excluding the historical records the Silver end-date logic produced, in line with the snapshot-only scope.

gold.fact_sales

- Takes the cleansed sales transactions and replaces the source business keys with the surrogate keys from the two dimensions, via left joins.
- Exposes the measures (sales amount, quantity, price) alongside the order, shipping, and due dates.

The result is a conventional star: a central fact surrounded by two conformed dimensions, joined exclusively on surrogate keys, ready for analytical querying.

6. Data Model

The Gold layer presents a star schema. The grain of each object is stated explicitly below, since grain is the foundation of correct analytical querying.

Object	Type	Grain
gold.dim_customers	Dimension	One row per unique customer.
gold.dim_products	Dimension	One row per current product (historical versions excluded).
gold.fact_sales	Fact	One row per sales order line item.

fact_sales links to dim_customers on customer_key and to dim_products on product_key. Both are surrogate keys generated in the Gold layer, which insulates the model from the differing key formats of the two source systems.

Full field-level definitions are maintained separately in the data catalog (docs/data_catalog.md).

7. Data Quality Issues Resolved

The following table consolidates the concrete data quality problems present in the raw sources and the treatment applied. These are the issues the transformation logic actually addresses, and form the substance of what the cleansing process contributes.

Issue in source	Where	Treatment
Duplicate customer records	crm_cust_info	Keep most recent per ID via ROW_NUMBER().
Null customer IDs	crm_cust_info	Excluded from load.
Untrimmed name whitespace	crm_cust_info	TRIM applied.
Cryptic gender / marital codes	crm_cust_info, erp_cust_az12	Expanded to readable values; unknowns → 'n/a'.
Dates stored as integers	crm_sales_details	Validated and cast to DATE; invalid → NULL.
Missing / inconsistent sales totals	crm_sales_details	Recalculated as quantity × price .
Invalid or missing price	crm_sales_details	Derived as sales ÷ quantity (NULLIF guard).
Missing product cost	crm_prd_info	Defaulted to 0.
Composite product key	crm_prd_info	Split into category ID and product key.
Future-dated birthdates	erp_cust_az12	Set to NULL.
Inconsistent ID formats (NAS prefix, dashes)	erp_cust_az12, erp_loc_a101	Normalised to match CRM keys.
Abbreviated / blank country codes	erp_loc_a101	Expanded; blanks → 'n/a'.
Conflicting gender across systems	Gold integration	CRM treated as system of record, ERP as fallback.

8. Validation & Testing

Validation in this project is applied at each layer in keeping with the medallion approach: completeness and schema checks after Bronze, correctness checks after Silver, and integration checks after Gold. The stored procedures additionally provide runtime safety through TRY...CATCH error reporting and per-step duration logging.

9. Future Work

- Build the analytics / BI layer: SQL reports and a dashboard covering customer behaviour, product performance, and sales trends.
- Add automated data quality tests as repeatable scripts rather than ad-hoc checks.
- Introduce incremental loading to replace full truncate-and-insert.
- Add slowly changing dimensions to preserve history, removing the current-snapshot limitation.
- Introduce orchestration (e.g. scheduling the load procedures) and basic CI on the SQL scripts.

10. Conclusion

This project delivers a working, end-to-end sales data warehouse: raw CSV extracts from two systems are ingested, cleansed and standardised through a documented set of transformations, and integrated into a star schema exposed as query-ready views. The layered medallion design keeps responsibilities cleanly separated, and the stored-procedure approach makes the pipeline reproducible.

Undertaken as a structured learning exercise, it provided hands-on experience across the core data engineering workflow — ETL, dimensional modelling, and data quality handling — and produced a documented, version-controlled artefact. The natural next step is the analytics layer, where the model built here begins to generate business insight.

Appendix: source scripts — `init_database.sql`, `ddl_bronze.sql`, `proc_load_bronze.sql`, `ddl_silver.sql`, `proc_load_silver.sql`, `ddl_gold.sql`. Supporting documentation — `README.md`, `data_catalog.md`, `naming-conventions.md`, `data_layers.md`.